✅ Lemma 1: β-rule for `nil`

c — Kód másolása

```
Lemma Listβ₁ {A T : Ty}
      {nilCase : Tm T}
      {consCase : Tm A -> Tm T -> Tm T} :
  iteList (A:=A) nilCase consCase nil = nilCase.
Proof.
  cbv [iteList nil].
  rewrite Indβ.
  cbv.
  rewrite Sumβ₁.
  reflexivity.
Defined.
```

Explanation:

- Expanding `iteList` in `cbv` reveals a call to `iteInd`.
- `Indβ` rewrites `iteInd f (con w)` into `f (map ... w)`.
- `inl tt` triggers `Sumβ₁`.

✅ Lemma 2: β-rule for `cons`

coq — Kód másolása

```
Lemma Listβ₂ {A T : Ty}
      {nilCase : Tm T}
      {consCase : Tm A -> Tm T -> Tm T}
      {x : Tm A} {xs : Tm (List A)} :
  iteList (A:=A) nilCase consCase (cons x xs)
  = consCase x (iteList nilCase consCase xs).
Proof.
  cbv [iteList cons].
  rewrite Indβ.
  cbv.
  rewrite Sumβ₂.
  rewrite Prodβ₁.
  rewrite Prodβ₂.
  reflexivity.
Defined.
```

Explanation:

- `cons x xs = con (inr (x, xs))`
- `Indβ` exposes `map` acting on `(x, xs)`
- `map` reduces via `Prodβ₁` and `Prodβ₂`
- `Sumβ₂` selects the right branch
- We recover the expected fold right equation.

Ezt mondtad:

Hey Chatty G! Given this context, can you help me define iteTree? The current definition doesn't seem to work. (* strictly positive types *) Inductive SPos : (Ty -> Ty) -> Type := | idP : SPos (fun X => X) | constP : forall (A : Ty), SPos (fun _ => A) | ProdP : forall {F G}, SPos F -> SPos G -> SPos (fun X => F X × G X) | SumP : forall {F G}, SPos F -> SPos G -> SPos (fun X => F X ⊎ G X) | ArrP : forall {F} (A : Ty), SPos F -> SPos (fun X => A ⇒ F X) . (* Coq also only allows strictly positive inductive types *) Fail Inductive X := | C : (X -> X) -> X. Fixpoint map {F : Ty -> Ty} {A B : Ty} (FP : SPos F) (f : Tm A -> Tm B) : Tm (F A) -> Tm (F B) := match FP in SPos F return Tm (F A) -> Tm (F B) with | idP => f | constP A => id | ProdP FP GP => fun t => (map FP f (fst t), map GP f (snd t)) | SumP FP GP => fun t => case t (fun u => inl (map FP f u)) (fun v => inr (map GP f v)) | ArrP A FP => fun t => lam (fun x => map FP f (t · x)) end. (* F (F (F (...))) — least fixpoint *) (* F (fix F) = fix F *) Parameter Ind : forall {F : Ty -> Ty}, SPos F -> Ty. Parameter con : forall {F : Ty -> Ty} (FP : SPos F), Tm (F (Ind FP)) -> Tm (Ind FP). Parameter iteInd : forall {F : Ty -> Ty} (FP : SPos F) {C : Ty}, (Tm (F C) -> Tm C) -> Tm (Ind FP) -> Tm C. Parameter Indβ : forall {F : Ty -> Ty} (FP : SPos F) {C : Ty} (f : Tm (F C) -> Tm C) (w : Tm (F (Ind FP))), iteInd FP f (con FP w) = f (map FP (iteInd FP f) w). (* |--------| |-----------| *) (* Tm (Ind FP) Tm (Ind FP) -> Tm C *) (* |--------------------| *) (* Tm (F (Ind FP)) -> Tm (F C) *) Parameter Indη : forall {F : Ty -> Ty} (FP : SPos F) {C : Ty} (f : Tm (F C) -> Tm C) (z : Tm (Ind FP) -> Tm C), (forall (w : Tm (F (Ind FP))), z (con FP w) = f (map FP z w)) -> z = iteInd FP f. (* binary trees: *) Definition BinTreeF : Ty -> Ty := fun X => ⊤ ⊎ (X × X). Lemma BinTreeP : SPos BinTreeF. Proof. cbv. apply SumP. apply constP. apply ProdP; apply idP. Defined. Definition BinTree := Ind BinTreeP. Definition leaf : Tm BinTree := con BinTreeP (inl tt). Definition node (l r : Tm BinTree) : Tm BinTree := con BinTreeP (inr (l, r)). Definition iteTree {T : Ty} (l : Tm T) (n : Tm T -> Tm T -> Tm T) (t : Tm BinTree) : Tm T := iteInd BinTreeP (fun x => case x (fun _ => l) (fun y z => n y z)) t.

A ChatGPT ezt mondta:

It **almost** works — the only thing wrong is the *shape of the algebra* you pass to iteInd.

🔍 The problem

Your datatype functor is

BinTreeF X = ⊤ ⊎ (X × X)

so an algebra for BinTreeF over a carrier T must have type:

Tm (BinTreeF T) → Tm T

≡ Tm (⊤ ⊎ (T × T)) → Tm T

That means that in the *right* branch of the case, the argument is not

y z : Tm T

but a single pair

p : Tm (T × T)

so your handler must first destruct that pair.
This is why

(fun x => case x (fun _ => l) (fun y z => n y z))

is rejected: a case on a sum has *one* argument for the inr-branch, not two.
y in the inr branch has type Tm (T × T), not two arguments.

✅ **Correct definition**

Here is a working definition, using fst/snd (as in your map):

Definition iteTree {T : Ty} (l : Tm T) (n : Tm T -> Tm T -> Tm T) (t : Tm BinTree) : Tm T := iteInd BinTreeP (fun x => case x (fun _ => l) (fun p => n (fst p) (snd p))) t.

Why this works

- In the leaf case: x = inl tt, so we return l.
- In the node case: x = inr (l', r'), so the branch argument p is (l', r') : Tm (T × T), and we feed both components to n.

This algebra has the correct shape:

f : Tm (⊤ ⊎ (T × T)) → Tm T

and therefore iteInd accepts it.

If you want, I can also help you prove iteTree leaf = l and iteTree (node a b) = n (iteTree a) (iteTree b) using Indβ.

Ezt mondtad:

Thank you! And given these definitions, how could I define a list that is empty? Let's call it nil here. (* lists: *) (* data List a = Nil | Cons a (List a) *) Definition ListF (A : Ty) : Ty -> Ty := fun X => ⊤ ⊎ (A × X). Lemma ListP (A : Ty) : SPos (ListF A). Proof. unfold ListF. apply SumP. apply constP. apply ProdP. apply constP. apply idP. Defined. Definition List (A : Ty) := Ind (ListP A).

A ChatGPT ezt mondta:

Your list functor is

ListF A X = ⊤ ⊎ (A × X)

so the empty list corresponds to the **left** injection (inl tt), exactly like your leaf for binary trees.

Given

Definition List (A : Ty) := Ind (ListP A).

the constructor is

con (ListP A) : Tm (ListF A (List A)) → Tm (List A)

and a value of type ListF A (List A) is

inl tt : the Nil branch

inr (a , xs) : the Cons branch

✅ **So** nil **is just:**

Definition nil {A : Ty} : Tm (List A) := con (ListP A) (inl tt).

(Optional) Also define cons

You'll probably also want:

Definition cons {A : Ty} (x : Tm A) (xs : Tm (List A)) : Tm (List A) := con (ListP A) (inr (x, xs)).

Now you have the list constructors matching:

Nil ≡ con (inl tt)

Cons a l ≡ con (inr (a , l))

If you want, I can also help define a list eliminator iteList analogous to your iteTree.

Ezt mondtad:

Thank you so much again! And could you help me write an iterator for this list definition which behaves like foldr in Haskell?

A ChatGPT ezt mondta:

Absolutely — the iterator for your list functor is *exactly* the same pattern as iteTree, just adapted to

ListF A X = ⊤ ⊎ (A × X)

so the algebra you pass to iteInd must have type

Tm (⊤ ⊎ (A × T)) → Tm T.

That corresponds to the usual Haskell foldr algebra:

nilCase : T

consCase : A → T → T

✅ iteList **(foldr)**

Definition iteList {A T : Ty} (nilCase : Tm T) (consCase : Tm A -> Tm T -> Tm T) (xs : Tm (List A)) : Tm T := iteInd (ListP A) (fun x => case x (fun _ => nilCase) (fun p => consCase (fst p) (snd p))) xs.

Explanation

- x : Tm (⊤ ⊎ (A × T))
- case x handles the two constructors:
    - inl tt → return nilCase
    - inr (a , t) → call consCase a t

🚀 **Expected behavior**

With

Definition nil {A} : Tm (List A) := con (ListP A) (inl tt). Definition cons {A} (x : Tm A) (xs : Tm (List A)) : Tm (List A) := con (ListP A) (inr (x, xs)).

You get the standard foldr equations from Indβ:

iteList nilCase consCase nil

= nilCase

iteList nilCase consCase (cons a xs)

= consCase a (iteList nilCase consCase xs)

If you'd like, I can also show how to define map, length, or append using iteList.

Ezt mondtad:

Thank you again! I have the two following beta rules for the tree's iterator: Lemma Treeβ_1 {T} {l : Tm T} {n : Tm T -> Tm T -> Tm T} : iteTree l n leaf = l. Proof. cbv. rewrite ! Indβ. cbv. rewrite ! Sumβ_1. reflexivity. Defined. Lemma Treeβ_2 {T} {l : Tm T} {n : Tm T -> Tm T -> Tm T} {t u : Tm BinTree} : iteTree l n (node t u) = n (iteTree l n t) (iteTree l n u). Proof. cbv. rewrite !

Indβ. cbv. rewrite ! $\text{Sum}\beta_2$. rewrite ! $\text{Prod}\beta_1$. rewrite ! $\text{Prod}\beta_2$. reflexivity. Defined. Could you help me write similar lemmas for the iterator of this list as well?

A ChatGPT ezt mondta:

Absolutely — the iteList iterator satisfies **exactly two β-rules** corresponding to Nil and Cons.
They follow the same pattern as your $\text{Tree}\beta_1$ and $\text{Tree}\beta_2$.

Below, I write them in the same style, assuming you already have the elimination rules:

- Indβ for Ind
- $\text{Sum}\beta_1$, $\text{Sum}\beta_2$ for ⊎
- $\text{Prod}\beta_1$, $\text{Prod}\beta_2$ for ×

Everything is structurally identical to the tree proofs.

📘 First: reminder of definitions

ListF A X = ⊤ ⊎ (A × X)

List A = Ind (ListP A)

nil = con (ListP A) (inl tt)

cons x xs = con (ListP A) (inr (x, xs))

and

Definition iteList {A T : Ty} (nilCase : Tm T) (consCase : Tm A -> Tm T -> Tm T) (xs : Tm (List A)) : Tm T := iteInd (ListP A) (fun x => case x (fun _ => nilCase) (fun p => consCase (fst p) (snd p))) xs.

✅ Lemma 1: β-rule for nil

Lemma $\text{List}\beta_1$ {A T : Ty} {nilCase : Tm T} {consCase : Tm A -> Tm T -> Tm T} : iteList (A:=A) nilCase consCase nil = nilCase. Proof. cbv [iteList nil]. rewrite Indβ. cbv. rewrite $\text{Sum}\beta_1$. reflexivity. Defined.

Explanation:

- Expanding iteList in cbv reveals a call to iteInd.
- Indβ rewrites iteInd f (con w) into f (map … w).

- inl tt triggers Sumβ_1.

✅ **Lemma 2: β-rule for** cons

Lemma Listβ_2 {A T : Ty} {nilCase : Tm T} {consCase : Tm A -> Tm T -> Tm T} {x : Tm A} {xs : Tm (List A)} : iteList (A:=A) nilCase consCase (cons x xs) = consCase x (iteList nilCase consCase xs). Proof. cbv [iteList cons]. rewrite Indβ. cbv. rewrite Sumβ_2. rewrite Prodβ_1. rewrite Prodβ_2. reflexivity. Defined.

Explanation:

- cons x xs = con (inr (x, xs))
- Indβ exposes map acting on (x, xs)
- map reduces via Prodβ_1 and Prodβ_2
- Sumβ_2 selects the right branch
- We recover the expected fold right equation.

🎉 **Resulting equations**

Your iteList now enjoys the usual foldr laws:

iteList nilCase consCase nil

= nilCase

iteList nilCase consCase (cons x xs)

= consCase x (iteList nilCase consCase xs)

Exactly as intended.

If you'd like, I can help you prove η-rules for lists (analogous to Indη), or define append, map, length, etc. using iteList.